

EPL Prediction Market Trading Strategy — Technical Specification

Technical Specification

Status: Draft v0.1 — for implementation by quant dev team

Mathematical expressions typeset natively in $\text{\LaTeX}$

August 2026

Contents

1	EPL Prediction Market Trading Strategy — Technical Specification	2
1.1	1. Purpose & Scope	2
1.2	2. Notation & Definitions	3
1.3	3. Module 1 — Club Value Model (Pre-Match Fundamental Value)	4
1.3.1	3.1 Player Value ‘ $V_{\{i,t\}}$ ’	4
1.3.2	3.2 Manager Value ‘ $M_{\{j,t\}}$ ’	4
1.3.3	3.3 Team Base Value	4
1.3.4	3.4 Contextual Adjustments	5
1.3.5	3.5 Composite Club Value	6
1.4	4. Module 2 — Match Outcome Probability Model	6
1.4.1	4.1 From value differential to a distribution	6
1.4.2	4.2 Fitting ‘m’ and ‘ σ ’	6
1.4.3	4.3 Heteroskedastic ‘ σ ’ (v1.1 extension, flagged not required for v1)	6
1.5	5. Module 3 — Season-Level Monte Carlo Simulation Engine	6
1.5.1	5.1 Algorithm	7
1.5.2	5.2 Path dependency is the important subtlety	7
1.5.3	5.3 Output	7
1.6	6. Module 4 — In-Play Value Model (Live Match Contracts)	7
1.6.1	6.1 Total in-play value	8
1.6.2	6.2 Time-decay of variance (required, not optional)	8
1.6.3	6.3 Live probability	8
1.7	7. Module 5 — Data Requirements & Sourcing	9
1.8	8. Module 6 — Calibration & Backtesting Framework	10
1.8.1	8.1 Historical dataset construction	10
1.8.2	8.2 Fitting procedure	10
1.8.3	8.3 Overfitting controls	11
1.9	9. Module 7 — Signal Generation & Trade Execution	11
1.9.1	9.1 Mispricing signal	11
1.9.2	9.2 Position sizing & PnL realization (as specified)	11
1.10	10. Module 8 — System Architecture	12
1.11	11. Summary of Explicit Judgment Calls Made in This Spec	13

1 EPL Prediction Market Trading Strategy — Technical Specification

Status: Draft v0.1 — for implementation by quant dev team **Scope:** Premier League football, prediction-market contracts (season outcomes, e.g. "top 4", and match outcomes, e.g. "X beats Y"), pre-match and in-play **Author's intent:** Exploit divergence between market-implied probability $p_{implied}$ and a model-derived "true" probability p_{true} —

1.1 1. Purpose & Scope

The strategy trades binary/categorical outcome contracts on football prediction markets (e.g. "Team X finishes top 4," "Team X beats Team Y") by:

1. Estimating a model probability p_{true} for each contract from a bottom-up club-strength model.
2. Comparing p_{true} to the market's $p_{implied}$.
3. Sizing and executing trades when a sufficient edge exists, using a defined position-sizing and PnL-realization rule.
4. Continuously recalibrating the model in-play as new information arrives (goals, subs, formation changes) for short-dated match contracts.

This document is organized into eight modules that map directly onto software components:

#	Module	Purpose
1	Club Value Model	Compute a scalar "strength" rating 'C' for each club, pre-match
2	Match Outcome Model	Convert a value differential into win/draw/loss probabilities
3	Season Simulation Engine	Monte Carlo simulate the rest of the season to price season-long contracts (top 4, title, relegation)
4	In-Play Value Model	Update club value in real time during a live match
5	Data Layer	Define what data is needed and where it comes from
6	Calibration & Back-testing	Fit and validate all model weights against history
7	Signal & Execution	Convert 'p_true' vs 'p_implied' into sized trades and PnL realization
8	System Architecture	How the above modules compose into a production pipeline

A note on rigor up front: several formulas in the original strategy sketch (e.g. "integrate a normal distribution from $-\infty$ to the value difference") are directionally correct but under-specified for implementation. This document tightens them into a standard, well-known class of model — an **ordered probit** on a latent value differential — which is a direct, implementable generalization of "integrate the normal CDF." This is called out explicitly wherever it happens so the dev team knows where a judgment call was made. —

1.2 2. Notation & Definitions

Symbol	Meaning
'X', 'Y'	The two teams in a fixture (X is the team we're pricing "from")
't'	Match index / time index (context-dependent)
'V_{i,t}'	Value of player 'i' heading into match 't'
'r_{i,t-k}'	Player 'i's performance rating in the match 'k' matches before 't'
'M_{j,t}'	Value of manager 'j' heading into match 't'
'C_X'	Composite fundamental club value of team 'X' (pre-match)
'D'	Value differential between two teams, 'D = C_X - C_Y'
'σ'	Standard deviation of the "noise" term in the outcome model (match-day variance not explained by 'D')
'Φ(z)'	Standard normal CDF
'_k'	Learned weight for factor 'k'
'p_true', 'p_implied'	Model probability and market-implied probability of a given outcome
'C_ingroup'	In-play adjustment to club value during a live match
'C_total'	'C_pre + C_ingroup', used for live win-probability updates

All weights ($\ast$, decay rates δ , the draw-margin m , and β) are **free parameters to be fit in Module 6**, not hand-set constants. Treat every formula below as a parametrized functional form, not a finished number. —

1.3 3. Module 1 — Club Value Model (Pre-Match Fundamental Value)

1.3.1 3.1 Player Value ‘V_{i,t}’

Exponentially-weighted average of the player’s last $N = 10$ match ratings, with non-participation scored as zero:

$$r_{i,t-k} = \text{performanceratingofplayeriinmatch}(t-k),$$

$$\text{if player did not play or played} < 10 \text{ minutes}$$

$$w_k = {}^k((0, 1), \text{e.g. } 0.85, \text{fit in backtest})$$

$$V_{i,t} = [\sum_{k=1..10} w_k \cdot r_{i,t-k}] / [\sum_{k=1..10} w_k]$$

Implementation notes:

- Use a fixed-length rolling window (deque) per player, re-weighted every match rather than re-computed from scratch.
- is a single global hyperparameter to start; consider letting it vary by position (GK/DEF/MID/FWD) if backtests show benefit — flag as a v2 extension, not v1 scope.
- Rating source: use a standardized performance-rating provider (see Module 5) rather than raw match stats, so that ”10.0 = best possible, 0 = did not play” is well-defined across positions.
- Injuries/suspensions are handled as a **separate** subtractive term (§3.4), not by zeroing $V_{i,t}$ — a player who is injured for the *upcoming* match still has a valid historical V , which is what should be removed from the team sum.

1.3.2 3.2 Manager Value ‘M_{j,t}’

Same exponential-decay structure, over the manager’s last 5 matches, rated on team-level performance (result vs. pre-match expectation, xG differential vs. actual, or similar):

$$M_{j,t} = [\sum_{k=1..5} {}^k \cdot s_{j,t-k}] / [\sum_{k=1..5} {}^k]$$

where $s_{j,t-k}$ is a team-performance score attributable to the manager (e.g. points-per-game relative to a pre-season Elo baseline) and k is a second decay hyperparameter, fit independently of k .

1.3.3 3.3 Team Base Value

$$C_{base_X,t} = \sum_{i \in \text{expectedXI}(X,t)} V_{i,t} + M_{j,t}$$

Key implementation decision: *expectedXI* must be defined explicitly, because the true starting XI for the upcoming match is unknown pre-kickoff. Two supported approaches:

1. **Expected-minutes weighting (recommended default):** for every player in the squad, maintain a rolling ”probability of starting / expected minutes share” i,t (derived from recent rotation patterns, press conference team-news, and injury status), and compute

$$C_{base_X,t} = \sum_{i \in \text{squad}(X)} i,t \cdot V_{i,t} + M_{j,t}$$

This avoids a discrete, brittle ”who starts” prediction and degrades gracefully when lineup news is unavailable far from kickoff.

1. **Confirmed-XI override:** once official lineups are published (60 min pre-kickoff), switch to the discrete sum over the confirmed 11 players. This is a strictly more accurate late-arriving signal and should override (1) when available — implement as a data-freshness-gated fallback, not an average of the two.

1.3.4 3.4 Contextual Adjustments

Each adjustment below is a signed additive term onto C_{base} . All coefficients are fit, not assumed: **(a) Team news (injuries / suspensions / red cards)**

$$C_{news_X, t} = - \sum_{i \text{ unavailable}(X, t)}^{baseline} V_{i, t}$$

Subtract the expected-minutes-weighted value of every unavailable player (red-card suspensions and confirmed injuries) relative to what §3.3 would otherwise have counted. If using approach (1) above, this is largely handled automatically by setting $i, t = 0$ for unavailable players — but keep it as an explicit auditable term in the pipeline for transparency and for approach (2), where a confirmed-XI omission needs to be distinguished from "rotated out" vs. "injured" (they may carry different signal for opponent in-game psychology, but functionally the value hit is the same). **(b) Home/away**

$$C_{venue_X, t} = +_{home} if X is home -_{away} if X is away (or a single symmetric home applied to home team only)$$

A single learned constant (in the same units as C), not a percentage — fit per-league; do not assume it is stable across seasons (crowd effects, travel, etc. can drift — re-fit periodically, see Module 6). **(c) Head-to-head (H2H)**

$$C_{h2h_X, t} = \sum_{k=1..K} \lambda^k \cdot h_{X, t-k}$$

where $h_{X, t-k}$ is a signed result-strength score for the k -th most recent meeting between X and Y specifically (e.g. +1/0/-1 for W/D/L, or goal-difference-based), λ is a decay parameter (fit separately from , — H2H effects likely decay differently than individual player form), and K is capped (e.g. last 10 meetings, or 5 years, whichever is fewer — cross-competition meetings, e.g. cup fixtures, are a config flag: include or exclude). **(d) Table position / matches remaining** This factor matters primarily for **season-long contracts** (Module 3) more than single-match win probability, but include it here as a small "context" term for match-level pricing too (e.g. dead-rubber effects, must-win desperation):

$$C_{table_X, t} = f(position_X, points_{gap} to target, matches_{remaining})$$

Model as a small number of hand-designed features (e.g. "points needed per remaining game to reach target," "already mathematically safe/out") passed through a fit linear or shallow non-linear term — do not hard-code the functional form until backtesting shows what shape it should take. Treat this as a *bias correction* layered on top of the ability-based value, not a leading factor. **(e) Transfer market** When a new player arrives (or an existing player departs), adjust the squad's V_i inputs directly rather than as a separate bolt-on adjustment:

$$V_{new_player, t} = average of new player's last 5 rated performances at previous club / competition, on a normalized ratings scale$$

Implementation notes:

- Cross-league rating normalization is required (a rating from Ligue 1 or the Championship is not on the same scale as the Premier League) — maintain a per-league scaling factor, fit against historical transfers where the player's rating is observable both before and after the move.
- New signings with no historical rating (e.g. very young players, different sport entirely — N/A here) get a provisional V from a squad-value/market-value proxy (see Module 5) until 5 PL performances accumulate.
- A newly-arrived player should also carry higher *uncertainty* than an established one — see §4.3 for how this feeds into , not just into C .

1.3.5 3.5 Composite Club Value

$$C_{X,t} = C_{base_{X,t}} + C_{news_{X,t}} + C_{venue_{X,t}} + C_{h2h_{X,t}} + C_{table_{X,t}}$$

(Transfer-market effects are already embedded in C_{base} via updated V_i , per 3.4(e).) All additive terms should be maintained as **separate named fields** in the data model (not pre-summed), so that Module 6 can attribute backtest error to individual components and re-weight them independently, and so any single term's coefficient going to zero doesn't require refactoring the pipeline. —

1.4 4. Module 2 — Match Outcome Probability Model

1.4.1 4.1 From value differential to a distribution

Define the pre-match value differential:

$$D = C_X - C_Y$$

The original spec calls for "a normal distribution, integrated from $-\infty$ to D " — implemented literally that only gives a two-outcome (win/not-win) probability. Football markets need three outcomes (win/draw/loss), so this is formalized as an **ordered probit model**, which is the standard textbook generalization of "integrate a Gaussian" to more than two ordered categories, and is directly compatible with the original intent: Let $m > 0$ be a fitted **draw margin** (how large a value edge is "not enough" to reliably show up as a win rather than a draw) and the fitted match-day noise SD. Then:

$$\begin{aligned} P(Xwins) &= 1 - \Phi((m - D)/) \\ P(draw) &= \Phi((m - D)/) - \Phi((-m - D)/) \\ P(Xloses) &= \Phi((-m - D)/) \end{aligned}$$

This collapses to the originally-described "integrate to D " formula when $m=0$ (pure two-outcome case), so it is a strict superset of the requested behavior, and it guarantees the three probabilities sum to 1 by construction (important — hand-rolled ad hoc formulas for win/draw/loss very often fail to sum to 1, which silently corrupts every downstream PnL calculation).

1.4.2 4.2 Fitting 'm' and 'σ'

Both are single global scalars (start there — do not add team-specific or fixture-specific until v1 is validated) fit by maximum likelihood on historical (D , actual result) pairs — see Module 6 for the estimation procedure. Do not hand-pick these values.

1.4.3 4.3 Heteroskedastic 'σ' (v1.1 extension, flagged not required for v1)

should shrink as more is known about a fixture (e.g. late team-news, no transfer-window uncertainty) and grow when key inputs are unusually uncertain (new signing with no PL track record, first meeting between two teams, manager in first 5 games). Recommended: $\sigma_t = \sigma_0 \cdot \exp(\sum_k u_{k,t})$ where u_k are uncertainty indicator features. This is not required for a working v1 but should be a clean extension point in the code (i.e. should be a function call, not a hardcoded constant, even in v1). —

1.5 5. Module 3 — Season-Level Monte Carlo Simulation Engine

Used to price contracts like "Chelsea finishes top 4," "Team X wins the title," "Team X is relegated."

1.5.1 5.1 Algorithm

```

for sim in 1..NSIMS (e.g. NSIMS = 100,000) :
    state = clone(current_league_table_and_club_values)
    foreach remaining_fixture(X, Y) in season_calendar, in date order :
        D = state.C[X] - state.C[Y]
        (pwin, pdraw, ploss) = ordered_prob(D, m, ) # Module 2
        result = sample_categorical([pwin, pdraw, ploss])
        update state.table with result(points, GD, GF)
    update state.C[X], state.C[Y] : # PATH - DEPENDENT see 5.2
    append simulated result to each team's rolling result/rating history
    recompute Vi, Cbase, Ch2h, etc. for the *next* fixture in this sim path
    record final standing(sim) for the team(s) of interest
    ptrue(top4) = (#sims where team finishes rank 1 - 4) / N

```

1.5.2 5.2 Path dependency is the important subtlety

Because V_i and C_{h2h} are functions of *recent match results*, a team's club value in gameweek 30 of a given simulation path depends on what happened in that path's simulated gameweeks 20-29 — which differs across simulation paths. This means:

- You cannot pre-compute a single static C_X for the rest of the season and reuse it across all simulations — C must be recomputed **within** each simulated path as results are drawn.
- For tractability, simulate at the level of **match result** (W/D/L, or a simple goal-difference draw) rather than full player-level match events. Feed that simulated result back into a simplified update of V_i (e.g. treat all players' next $r_{i,t}$ as a single team-level "implied performance rating" derived from the simulated result and expected-vs-actual goal difference, rather than re-simulating individual player ratings). This is a deliberate simplification — flag it as such in code comments, since it is the single biggest approximation in the season-simulation module.
- Player availability (injuries, suspensions) for *future* simulated fixtures should be modeled probabilistically too (e.g. a base injury hazard rate per player, plus deterministic suspension triggers from simulated red/yellow cards) rather than frozen at today's team-news — otherwise the simulation understates variance in run-ins.

1.5.3 5.3 Output

The simulation should output, per team, per simulation batch:

- Full distribution over final league position (1st through 20th)
- Point estimate + confidence interval for any position-threshold market (top 4, top 6, bottom 3, etc.)
- These feed directly into p_{true} for Module 7.

Re-run nightly (or after any material news event — red card, injury, big result) rather than only once per gameweek, since $p_{implied}$ in the market moves continuously and stale simulations create false signals. —

1.6 6. Module 4 — In-Play Value Model (Live Match Contracts)

For "will X win this specific, currently-in-progress match" contracts.

1.6.1 6.1 Total in-play value

$$C_{totalX}(t) = C_{preX} + C_{ingameX}(t)$$

C_{preX} is the Module 1 output, frozen at kickoff. $C_{ingameX}(t)$ accumulates during the match:

$$\begin{aligned} C_{ingameX}(t) = & \textit{score} \cdot (G_X(t) - G_Y(t)) \\ & + \textit{form} \cdot \textit{formation}_{termX}(t) \\ & + \sum_{\textit{subsmade}} [\textit{subswap} \cdot (V_{on} - V_{off}) + \textit{subtactical} \cdot \textit{tactical}_{bonus}(\textit{subcontext})] \end{aligned}$$

Notes per term:

- **Scoreline** ($\textit{score}$): signed goal difference *at time t^* , not final — recompute every time the scoreline changes. Note this term will typically need a *time-remaining* interaction (a 1-goal lead with 85 minutes gone is worth far more than a 1-goal lead at minute 10) — see §6.2, this is not optional for a usable in-play model.
- **Formation** ($\textit{form}$): a categorical/ordinal signal — e.g. "went from 4-3-3 to 5-4-1 while leading" → positive $\textit{formation}_{term}$; "went more attacking while trailing" → the sign depends on whether it's modeled as increasing X's own value or decreasing the opponent's effective defensive value. Recommend implementing this as a symmetric feature: $\textit{formation}_{termX}(t) = \textit{shape}_{change_{score}}(X's \textit{formation}_{before}, X's \textit{formation}_{after}, \textit{game}_{state})$, fit empirically rather than hand-signed, since "go defensive when leading" does not universally help (some teams defend a lead worse than they attack). This is explicitly a candidate for a learned rather than hand-set sign/magnitude.
- **Subs** ($\textit{subswap}$, $\textit{subtactical}$): V_{on} and V_{off} use the same Module 1 player-value definition. The $\textit{tactical}_{bonus}$ term is a separate, smaller learned adjustment capturing *timing* (a sub at minute 60 while level carries different signal than a sub at minute 88 while leading) — implement as a function of $(\textit{game}_{minute}, \textit{game}_{state}_{at_{sub}})$.
- **H2H and home/away**: carried through unchanged from C_{pre} (they don't change during the match) — do not re-derive them here, they are already inside C_{preX} .

1.6.2 6.2 Time-decay of variance (required, not optional)

As the match clock runs down, outcome uncertainty shrinks mechanically (fewer minutes left for the score to change), independent of anything in C_{ingame} . Model:

$$(t) = 0 \cdot g(\textit{minutes}_{remaining})$$

with g a decreasing function of time remaining (e.g. $g() = \textit{sqrt}(/90)$) as a starting functional form, to be fit against realized in-play result frequencies — this is directly analogous to time-decay in options-pricing-style live win probability models used across other sports, and is *necessary* for the in-play model to be well-calibrated near full time; without it the model will be badly overconfident early and badly underconfident late, or vice versa, depending on how $\textit{score}$ was fit).

1.6.3 6.3 Live probability

Re-run the exact same ordered-probit formula from Module 2, substituting $DC_{totalX}(t) - C_{totalY}(t)$ and (t) :

$$P(X \textit{wins} | t) = 1 - \Phi((m - D_{total}(t)) / (t))$$

Recompute on every scoreline change, every substitution, and every formation-change event, plus on a fixed heartbeat (e.g. every 60 seconds) to capture the pure time-decay effect even when nothing else has happened. —

1.7 7. Module 5 — Data Requirements & Sourcing

Data	Used in	Candidate sources
Player performance ratings, per match, per player	§3.1	Provider with standardized match ratings (e.g. Opta/StatsPerform, WhoScored, SofaScore feeds) — need a licensed feed for production reliability, not scraped
Minutes played per player per match	§3.1	Same as above, or official match-center data
Manager tenure & team results	§3.2	League official results feed + a manager-change dataset
League table, points, GD, matches remaining	§3.4(d), §5	Official league data feed (e.g. football-data.org, Premier League API)
Historical head-to-head results	§3.4(c)	Official results archive (e.g. football-data.co.uk historical CSVs, or the same provider as the table feed)
Injuries / suspensions / team news	§3.4(a)	Dedicated injury-tracking feed (e.g. PhysioRoom, Premier Injuries) + press-conference/team-news scraping as a secondary confirmation source
Transfer market activity & player valuations	§3.4(e)	Transfermarkt (market value as a *prior* for new/unrated players), plus the same performance-rating provider for pre-move history
Live scoreline, formation, substitutions, cards	§6	Live/in-play sports data feed (e.g. Sportradar, Genius Sports, Opta live) — this needs low-latency delivery, unlike the pre-match modules
Market-implied prices ‘p_implied’	§8	The prediction market’s own order book / last-traded-price API

check realized win frequency against predicted, across many historical matches with full event timelines.

1.8.3 8.3 Overfitting controls

- Regularize the weight vector (L2 penalty) given the number of free parameters relative to the number of matches per season (380).
- Hold out at least the most recent 1-2 seasons as a true, never-touched-during-fitting final test set.
- Re-fit on a fixed schedule (e.g. every close season) rather than continuously online, to avoid the model quietly drifting in response to short streaks; log every re-fit's parameter values and out-of-sample metrics for audit.

1.9 9. Module 7 — Signal Generation & Trade Execution

1.9.1 9.1 Mispricing signal

For every open contract on the target market:

$$edge = p_{true} - p_{implied}$$

Only proceed to sizing if $|edge|$ exceeds a minimum threshold (fit or chosen to cover expected transaction costs/spread — do not trade every non-zero edge).

1.9.2 9.2 Position sizing & PnL realization (as specified)

Using the notation from the original theory sheet, with k a position-sizing scale constant:

- If $p_{true} < p_{implied}$ (market overpricing "Yes" / underpricing "No"):
- Buy $k \cdot (p_{implied} - p_{true})$ units of **No** shares.
- When the market subsequently converges so that $p_{implied} = p_{true}$, sell those No shares. Realized PnL on that leg:

$$PnL = k \cdot (p_{implied_{entry}} - p_{true})$$

- If $p_{true} > p_{implied}$ (market underpricing "Yes"):
- Buy $k \cdot (p_{true} - p_{implied})$ units of **Yes** shares (analogous exit/PnL logic applies symmetrically on convergence).

Implementation flags for the dev team (these are risk/design points, not changes to the stated formula):

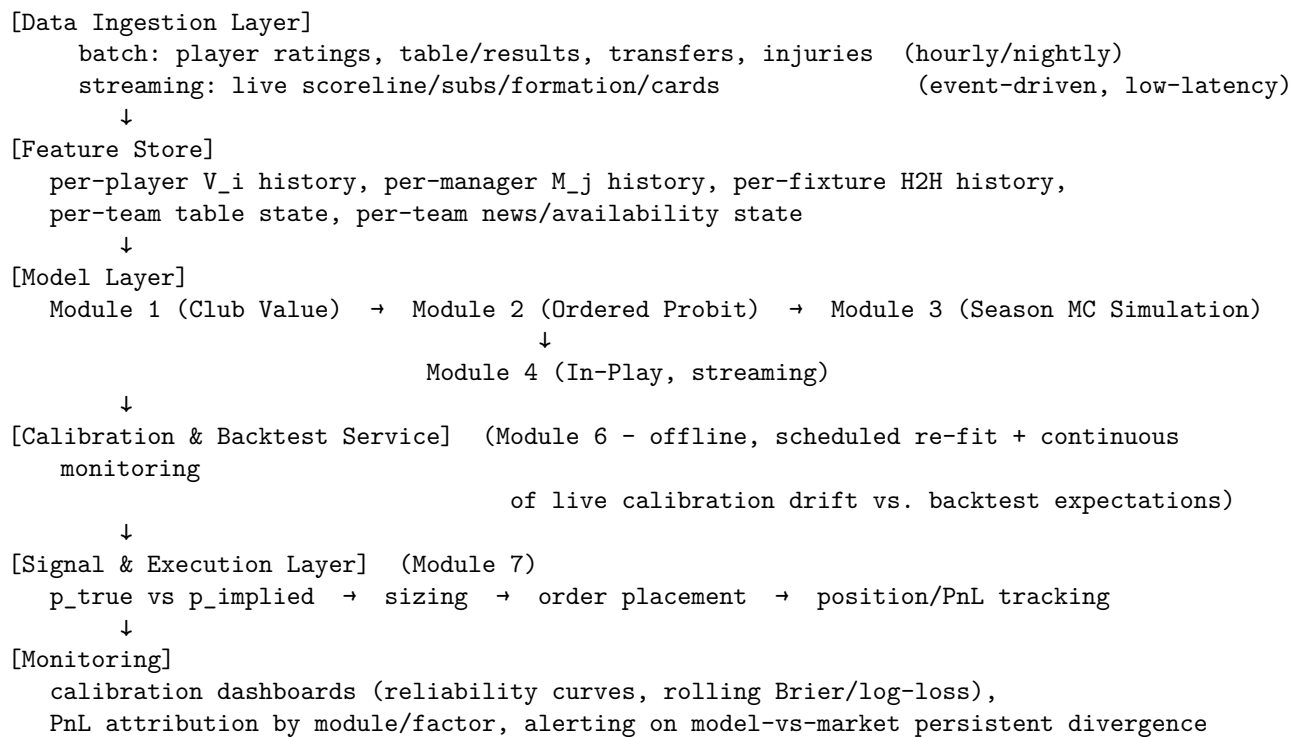
1. ****This sizing rule is quadratic in the mispricing, and realized PnL is cubic in the *entry* mispricing.**** That means small edges are sized very conservatively and large edges very aggressively — verify this is the intended risk profile (it is **not** Kelly-optimal in the classical sense; it's closer to a "size like a market maker who wants a lot of size only when very confident" heuristic). Recommend backtesting sensitivity to k explicitly, and capping position size independent of the formula (a hard max-position-per-contract and max-aggregate-exposure-per-team/per-gameweek), since the formula alone has no notion of bankroll or liquidity.
2. **The PnL formula assumes convergence happens ($p_{implied} = p_{true}$) before market resolution or before the edge reverses.** In practice: (a) the market may never converge before it resolves (settlement PnL should be computed from the actual binary outcome and entry price, not from the convergence formula, if resolution happens first); (b) p_{true} itself moves over time as news arrives (a new injury, a red card) — re-derive p_{true} continuously and treat the formula's

p_{true} as "current model estimate at the time of marking," not a fixed target. Build the exit logic as: exit on convergence-to-current-model-estimate **or** on contract resolution, **or** on a stop-loss if the gap widens materially and stays wide beyond a re-validation window (suggesting the model, not the market, is wrong) — this last condition is a risk control not in the original sketch and should be added.

3. **Transaction costs / spread / slippage** are not in the stated formula — subtract expected round-trip cost from the edge threshold in §9.1 and from the PnL realized in backtest, or the backtest will overstate live performance.
4. **Model-risk sizing:** since p_{true} is itself an estimate with uncertainty (compounded from Module 1 parameter uncertainty, Module 2/3 simulation noise, and data staleness), consider scaling k down when the underlying simulation's p_{true} estimate has wide Monte Carlo standard error (a natural output of Module 3) or when t inputs (§4.3, §6.2) indicate an unusually uncertain fixture — i.e. two contracts with the same nominal edge should not necessarily get the same size if one is far better-supported by data than the other.

1.10 10. Module 8 — System Architecture

Suggested service decomposition:



Key non-functional requirements to plan for explicitly:

- **Idempotent, replayable feature computation** — since backtesting requires reconstructing exact historical feature values as-of a timestamp, the same feature-computation code path should run in both backtest and live modes (avoid maintaining two parallel implementations that can silently diverge).
- **Separate deploy/scale cadence** for the batch pre-match modules vs. the streaming in-play module, per §7 note.
- **Full audit logging** of every p_{true} estimate, its input feature values, and the model/parameter version that produced it, tied to every trade — required both for the risk controls in §9.2 and for post-hoc debugging of calibration drift.

1.11 11. Summary of Explicit Judgment Calls Made in This Spec

For traceability, since this spec formalizes some under-specified parts of the original sketch:

1. Win/draw/loss modeled as an **ordered probit** (draw margin m + normal noise) rather than a bare normal-CDF integral, so that three outcomes are handled and probabilities sum to 1.
2. Starting XI handled via **expected-minutes weighting** with a confirmed-lineup override, rather than requiring a discrete lineup prediction.
3. Season Monte Carlo simulation is explicitly **path-dependent** (club values evolve within each simulated path) — flagged as the single largest source of approximation, since full player-level in-path resimulation is likely intractable at 100k+ simulation scale.
4. In-play model requires a **time-decaying variance term** (t), not mentioned in the original sketch, without which the live probability model will be systematically miscalibrated.
5. Added a **stop-loss / model-disagreement** exit condition and a **transaction-cost floor** to the trade execution logic, since the original PnL formulas assume convergence and ignore costs.

Everything else (player-value decay structure, manager value, H2H decay, home/away constant, transfer-market handling, the quadratic sizing / cubic PnL rule) is implemented as originally specified, with hyperparameters left explicitly open for Module 6 to fit rather than hardcoded.